Python高阶技巧

黑马程序员
www.itheima.com
传智教育旗下
高端IT教育品牌

目录

Contents

- 闭包
- 装饰器
- 设计模式
- 多线程
- 网络编程
- 正则表达式
- 递归

学习目标

Learning Objectives

1. 掌握闭包的用途和用法
2. 掌握nonlocal关键字的作用

闭包

```
account_amount = 0        # 账户余额
def atm(num, deposit=True):
    global account_amount
    if deposit:
        account_amount += num
        print(f"存款:+{num}，账户余额:{account_amount}")
    else:
        account_amount -= num
        print(f"取款:-{num}，账户余额:{account_amount}")


atm(300)
atm(300)
atm(100, False)
```

```
D:\dev\Python\Python310\
存款:+300，账户余额:300
存款:+300，账户余额:600
取款:-100，账户余额:500
```

通过全局变量account_amount来记录余额

尽管功能实现是ok的，但是仍有问题：

- 代码在命名空间上（变量定义）不够干净、整洁
- 全局变量有被修改的风险

如何解决？

- 将变量定义在函数内部是行不通的
- 我们需要使用闭包

闭包

在函数嵌套的前提下，内部函数使用了外部函数的变量，并且外部函数返回了内部函数，我们把这个使用外部函数变量的内部函数称为闭包。

```python
account_amount = 0        # 账户余额
def atm(num, deposit=True):
    global account_amount
    if deposit:
        account_amount += num
        print(f"存款:+{num}，账户余额:{account_amount}")
    else:
        account_amount -= num
        print(f"取款:-{num}，账户余额:{account_amount}")


atm(300)
atm(300)
atm(100, False)
```

```python
def account_create(initial_amount=0):
    def atm(num, deposit=True):
        nonlocal initial_amount
        if deposit:
            initial_amount += num
            print(f"存款:+{num}，账户余额:{initial_amount}")
        else:
            initial_amount -= num
            print(f"取款:-{num}，账户余额:{initial_amount}")

    return atm


fn = account_create()
fn(300)
fn(200)
fn(300, False)
```

```
D:\dev\Python\Python31
存款:+300，账户余额:300
存款:+200，账户余额:500
取款:-300，账户余额:200
```

闭包

简单闭包

```
def outer(logo):

    def inner(msg):
        print(f"<{logo}>{msg}<{logo}>")

    return inner


fn1 = outer("黑马程序员")
fn1("大家好呀")
fn1("学Python就来")

fn2 = outer("传智教育")
fn2("IT职业教育培训")
fn2("学Python就来")
```

外部函数

内部函数

返回内部函数

fn1，就是确定了logo变量为 黑马程序员 的inner函数
fn2，就是确定了logo变量为 传智教育 的inner函数

```
<黑马程序员>大家好呀<黑马程序员>
<黑马程序员>学Python就来<黑马程序员>
<传智教育>IT职业教育培训<传智教育>
<传智教育>学Python就来<传智教育>
```

闭包

修改外部函数变量的值

```
def outer(num1):

    def inner(num2):
        nonlocal num1
        num1 += num2
        print(num1)

    return inner


fn = outer(10)
fn(10)
fn(10)
```

需要使用nonlocal关键字修饰外部函数的变量
才可在内部函数中修改它

闭包

现在可以尝试实现以下atm取钱的闭包实现了

```python
account_amount = 0        # 账户余额
def atm(num, deposit=True):
    global account_amount
    if deposit:
        account_amount += num
        print(f"存款:+{num}，账户余额:{account_amount}")
    else:
        account_amount -= num
        print(f"取款:-{num}，账户余额:{account_amount}")


atm(300)
atm(300)
atm(100, False)
```

```python
def account_create(initial_amount=0):
    def atm(num, deposit=True):
        nonlocal initial_amount
        if deposit:
            initial_amount += num
            print(f"存款:+{num}，账户余额:{initial_amount}")
        else:
            initial_amount -= num
            print(f"取款:-{num}，账户余额:{initial_amount}")

    return atm


fn = account_create()
fn(300)
fn(200)
fn(300, False)
```

```
D:\dev\Python\Python31
存款:+300，账户余额:300
存款:+200，账户余额:500
取款:-300，账户余额:200
```

闭包注意事项

优点，使用闭包可以让我们得到：

- 无需定义全局变量即可实现通过函数，持续的访问、修改某个值
- 闭包使用的变量的所用于在函数内，难以被错误的调用修改

缺点：

- 由于内部函数持续引用外部函数的值，所以会导致这一部分内存空间不被释放，一直占用内存

总结

1. 什么是闭包

定义双层嵌套函数， 内层函数可以访问外层函数的变量

将内存函数作为外层函数的返回，此内层函数就是闭包函数

2. 闭包的好处和缺点

- 优点：不定义全局变量，也可以让函数持续访问和修改一个外部变量
- 优点：闭包函数引用的外部变量，是外层函数的内部变量。作用域封闭难以被误操作修改
- 缺点：额外的内存占用

3. nonlocal关键字的作用

在闭包函数（内部函数中）想要修改外部函数的变量值

需要用nonlocal声明这个外部变量

目录

Contents

- 闭包
- 装饰器
- 设计模式
- 多线程
- 网络编程
- 正则表达式
- 递归

学习目标

Learning Objectives

1. 掌握装饰器的作用和用法

装饰器

装饰器其实也是一种闭包， 其功能就是在不破坏目标函数原有的代码和功能的前提下，为目标函数增加新功能。

```
def sleep():
    import random
    import time
    print("睡眠中......")
    time.sleep(random.randint(1, 5))
```

希望给sleep函数，增加一个功能：

- 在调用sleep前输出：我要睡觉了
- 在调用sleep后输出：我起床了

```
print("我要睡觉了")
sleep()
print("我起床了")
```

可以使用装饰器来实现

装饰器的一般写法（闭包写法）

```
def outer(func):
    def inner():
        print("我要睡觉了")
        func()
        print("我起床了")

    return inner


def sleep():
    import random
    import time
    print("睡眠中......")
    time.sleep(random.randint(1, 5))


fn = outer(sleep)
fn()
```

定义一个闭包函数， 在闭包函数内部：

- 执行目标函数
- 并完成功能的添加

执行结果：

```
我要睡觉了
睡眠中......
我起床了
```

装饰器的语法糖写法

```
def outer(func):
    def inner():
        print("我要睡觉了")
        func()
        print("我起床了")

    return inner


@outer
def sleep():
    import random
    import time
    print("睡眠中......")
    time.sleep(random.randint(1, 5))


sleep()
```

使用@outer

定义在目标函数sleep之上

执行结果：

```
我要睡觉了
睡眠中......
我起床了
```

总结

1. 什么是装饰器

装饰器就是使用创建一个闭包函数，在闭包函数内调用目标函数。

可以达到不改动目标函数的同时，增加额外的功能。

2. 装饰器的写法

```
def outer(func):
    def inner():
        print("我要睡觉了")
        func()
        print("我起床了")

    return inner


@outer
def sleep():
    import random
    import time
    print("睡眠中......")
    time.sleep(random.randint(1, 5))


sleep()
```

目录

Contents

- 闭包
- 装饰器
- 设计模式
  - 单例模式
  - 工厂模式
- 多线程
- 网络编程
- 正则表达式
- 递归

学习目标

Learning Objectives

1. 掌握单例模式的作用和写法

设计模式

设计模式是一种编程套路，可以极大的方便程序的开发。

最常见、最经典的设计模式，就是我们所学习的面向对象了。

除了面向对象外，在编程中也有很多既定的套路可以方便开发，我们称之为设计模式：

- 单例、工厂模式
- 建造者、责任链、状态、备忘录、解释器、访问者、观察者、中介、模板、代理模式
- 等等模式

设计模式非常多，我们主要挑选了2个经常用到的进行讲解。

单例模式

```
class Tool:
    pass


t1 = Tool()
t2 = Tool()
print(t1)
print(t2)
```

创建类的实例后，就可以得到一个完整的、独立的类对象。

通过print语句可以看出，它们的内存地址是不相同的，即t1和t2是完全独立的两个对象。

```
<__main__.Tool object at 0x0000027246FCCEE0>
<__main__.Tool object at 0x0000027246FCD9F0>
```

某些场景下， 我们需要一个类无论获取多少次类对象，都仅仅提供一个具体的实例

用以节省创建类对象的开销和内存开销

比如某些工具类，仅需要1个实例，即可在各处使用

这就是单例模式所要实现的效果。

单例模式

单例模式（Singleton Pattern）是一种常用的软件设计模式，该模式的主要目的是确保某一个类只有一个实例存在。在整个系统中，某个类只能出现一个实例时，单例对象就能派上用场。

- 定义：保证一个类只有一个实例，并提供一个访问它的全局访问点
- 适用场景：当一个类只能有一个实例，而客户可以从一个众所周知的访问点访问它时。

单例的实现模式：

```
class StrTools:
    pass


str_tool = StrTools()
```

在一个文件中定义如上代码

```
from test import str_tool

s1 = str_tool
s2 = str_tool
print(s1)
print(s2)
```

在另一个文件中导入对象

```
<test.StrTools object at 0x0000013D001DB910>
<test.StrTools object at 0x0000013D001DB910>
```

s1和s2是同一个对象

总结

1. 什么是设计模式

设计模式就是一种编程套路。

使用特定的套路得到特定的效果

2. 什么是单例设计模式

单例模式就是对一个类，只获取其唯一的类实例对象，持续复用它。

- 节省内存
- 节省创建对象的开销

目录

Contents

- 闭包
- 装饰器
- 设计模式
  - 单例模式
  - 工厂模式
- 多线程
- 网络编程
- 正则表达式
- 递归

学习目标

Learning Objectives

1. 掌握工厂模式的作用和写法

工厂模式

当需要大量创建一个类的实例的时候， 可以使用工厂模式。

即，从原生的使用类的构造去创建对象的形式

迁移到，基于工厂提供的方法去创建对象的形式。

```
class Person:
    pass

class Worker(Person):
    pass
class Student(Person):
    pass
class Teacher(Person):
    pass

worker = Worker()
stu = Student()
teacher = Teacher()
```

```
class Person:
    pass

class Worker(Person):
    pass
class Student(Person):
    pass
class Teacher(Person):
    pass

class Factory:
    def get_person(self, p_type):
        if p_type == 'w':
            return Worker()
        elif p_type == 's':
            return Student()
        else:
            return Teacher()

factory = Factory()
worker = factory.get_person('w')
stu = factory.get_person('s')
teacher = factory.get_person('t')
```

工厂模式

```
class Person:
    pass

class Worker(Person):
    pass
class Student(Person):
    pass
class Teacher(Person):
    pass

class Factory:
    def get_person(self, p_type):
        if p_type == 'w':
            return Worker()
        elif p_type == 's':
            return Student()
        else:
            return Teacher()

factory = Factory()
worker = factory.get_person('w')
stu = factory.get_person('s')
teacher = factory.get_person('t')
```

- 使用工厂类的get_person()方法去创建具体的类对象

优点：

- 大批量创建对象的时候有统一的入口，易于代码维护
- 当发生修改，仅修改工厂类的创建方法即可
- 符合现实世界的模式，即由工厂来制作产品（对象）

总结

1. 什么是工厂模式

将对象的创建由使用原生类本身创建

转换到由特定的工厂方法来创建

2. 好处

- 大批量创建对象的时候有统一的入口，易于代码维护
- 当发生修改，仅修改工厂类的创建方法即可
- 符合现实世界的模式，即由工厂来制作产品（对象）

目录

Contents

- 闭包
- 装饰器
- 设计模式
- 多线程
  - 进程、线程和并行执行
  - 多线程编程
- 网络编程
- 正则表达式
- 递归

学习目标

Learning Objectives

1. 了解什么是进程、线程
2. 了解什么是并行执行

进程、线程

现代操作系统比如Mac OS X，UNIX，Linux，Windows等，都是支持“多任务”的操作系统。

进程： 就是一个程序，运行在系统之上，那么便称之这个程序为一个运行进程，并分配进程ID方便系统管理。

线程：线程是归属于进程的，一个进程可以开启多个线程，执行不同的工作，是进程的实际工作最小单位。

进程就好比一家公司，是操作系统对程序进行运行管理的单位

线程就好比公司的员工，进程可以有多个线程（员工），是进程实际的工作者

操作系统中可以运行多个进程，即多任务运行

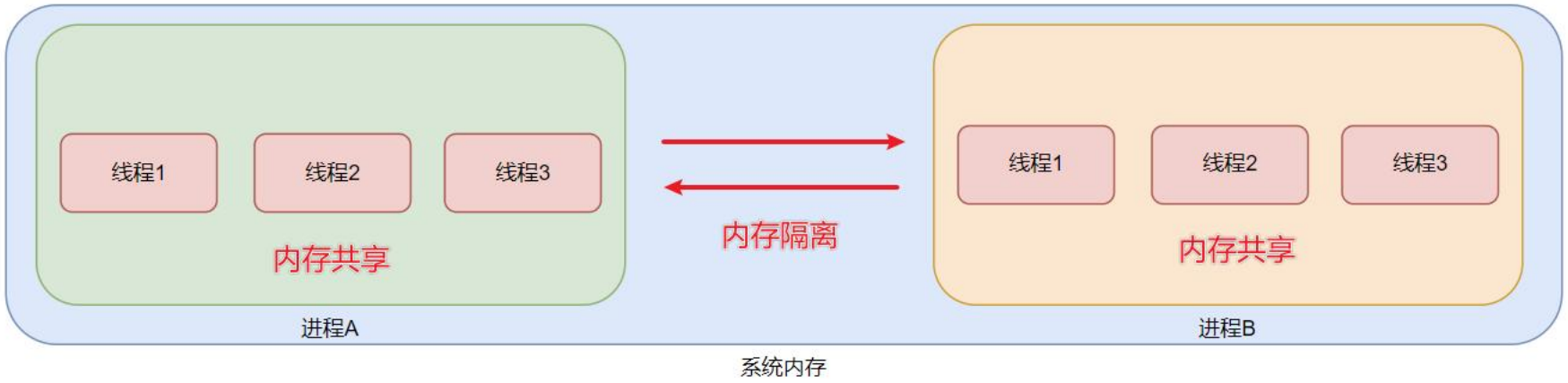
一个进程内可以运行多个线程，即多线程运行

进程、线程

注意点：

进程之间是内存隔离的， 即不同的进程拥有各自的内存空间。 这就类似于不同的公司拥有不同的办公场所。

线程之间是内存共享的，线程是属于进程的，一个进程内的多个线程之间是共享这个进程所拥有的内存空间的。这就好比，公司员工之间是共享公司的办公场所。



并行执行

并行执行的意思指的是同一时间做不同的工作。

进程之间就是并行执行的，操作系统可以同时运行好多程序，这些程序都是在并行执行。

除了进程外，线程其实也是可以并行执行的。

也就是比如一个Python程序，其实是完全可以做到：

- 一个线程在输出：你好
- 一个线程在输出：Hello

像这样一个程序在同一时间做两件乃至多件不同的事情， 我们就称之为：多线程并行执行

总结

1. 什么是进程

程序在操作系统内运行，即成为一个运行进程

2. 什么是线程

进程内部可以有多个线程，程序的运行本质上就是由进程内部的线程在实际工作的。

3. 什么是并行执行

- 多个进程同时在运行，即不同的程序同时运行，称之为：多任务并行执行
- 一个进程内的多个线程同时在运行，称之为：多线程并行执行

目录

Contents

- 闭包
- 装饰器
- 设计模式
- 多线程
  - 进程、线程和并行执行
  - 多线程编程
- 网络编程
- 正则表达式
- 递归

学习目标

Learning Objectives

1. 掌握使用threading模块完成多线程编程

threading模块

绝大多数编程语言，都允许多线程编程，Pyhton也不例外。

Python的多线程可以通过threading模块来实现。

```
import threading

thread_obj = threading.Thread([group [, target [, name [, args [, kwargs]]]]])
- group: 暂时无用，未来功能的预留参数
- target: 执行的目标任务名
- args: 以元组的方式给执行任务传参
- kwargs: 以字典方式给执行任务传参
- name: 线程名，一般不用设置
```

```
# 启动线程，让线程开始工作
thread_obj.start()
```

多线程编程

```
def sing():
    while True:
        print("我在唱歌，啦啦啦...")
        time.sleep(1)


def dance():
    while True:
        print("我在跳舞，哗哗哗哗哗")
        time.sleep(1)
```

```
"""多线程运行模式"""
sing_thread = threading.Thread(target=sing)
dance_thread = threading.Thread(target=dance)

sing_thread.start()
dance_thread.start()
```

通过左侧代码

即可实现多线程编程。

让一个Python程序实现启动2个线程

每个线程各自执行一个函数

多线程编程

```python
def sing():
    while True:
        print("我在唱歌，啦啦啦...")
        time.sleep(1)


def dance(msg):
    while True:
        print(msg)
        time.sleep(1)
"""多线程运行模式"""
sing_thread = threading.Thread(target=sing)
dance_thread = threading.Thread(target=dance, args=("我在跳舞，哈哈哈", ))

sing_thread.start()
dance_thread.start()
"""多线程运行模式"""
sing_thread = threading.Thread(target=sing)
dance_thread = threading.Thread(target=dance, kwargs={"msg": "我在跳舞，哈哈哈"})

sing_thread.start()
dance_thread.start()
```

需要传参的话可以通过：

- args参数通过元组（按参数顺序）的方式传参
- 或使用kwargs参数用字典的形式传参

总结

1. threading模块的使用

thread_obj = threading.Thread(target=func) 创建线程对象

thread_obj.start() 启动线程执行

2. 如何传参

```
"""多线程运行模式"""
sing_thread = threading.Thread(target=sing)
dance_thread = threading.Thread(target=dance, args=("我在跳舞，哈哈哈", ))

sing_thread.start()
dance_thread.start()
```

```
"""多线程运行模式"""
sing_thread = threading.Thread(target=sing)
dance_thread = threading.Thread(target=dance, kwargs={"msg": "我在跳舞，哈哈哈"})

sing_thread.start()
dance_thread.start()
```

练习 多线程练习案例 -

目录

Contents

- 闭包
- 装饰器
- 设计模式
- 多线程
- 网络编程
  - 服务端开发
  - 客户端开发
- 正则表达式
- 递归

学习目标

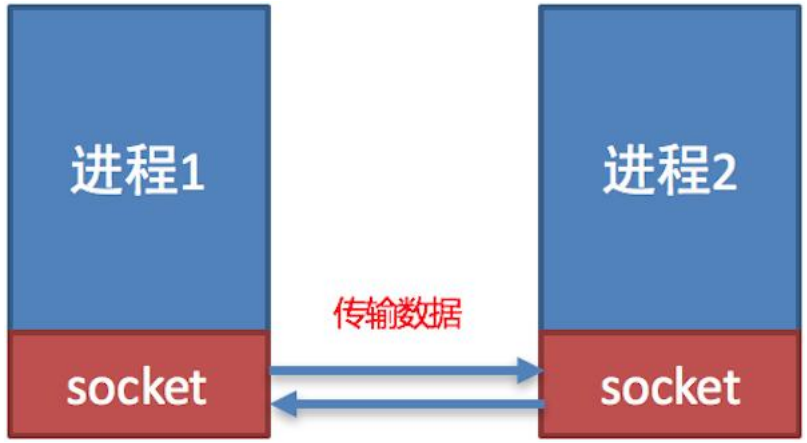
Learning Objectives

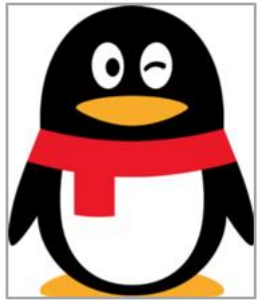
1. 了解什么是Socket网络编程
2. 基于Socket完成服务端程序开发

Socket

socket（简称 套接字）是进程之间通信一个工具，好比现实生活中的插座，所有的家用电器要想工作都是基于插座进行，进程之间想要进行网络通信需要socket。

Socket负责进程之间的网络数据传输，好比数据的搬运工。





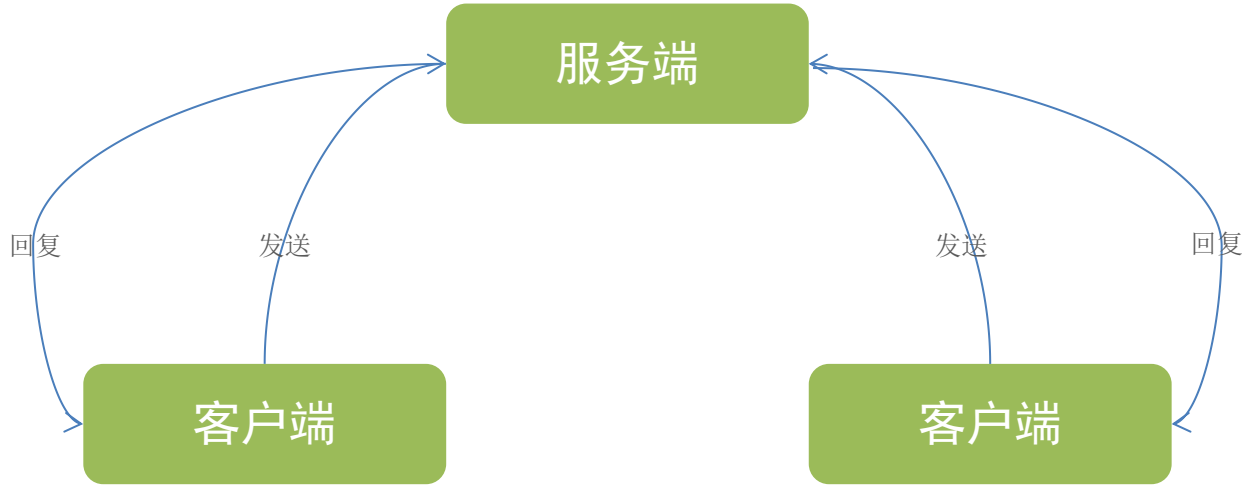
大多数软件都使用到了Socket进行网络通讯

客户端和服务端

2个进程之间通过Socket进行相互通讯，就必须有服务端和客户端

Socket服务端：等待其它进程的连接、可接受发来的消息、可以回复消息

Socket客户端：主动连接服务端、可以发送消息、可以接收回复



Socket服务端编程

主要分为如下几个步骤：

1. 创建socket对象

```
import socket
socket_server = socket.socket()
```

2. 绑定socket_server到指定IP和地址

```
socket_server.bind(host, port)
```

3. 服务端开始监听端口

```
socker_server.listen(backlog)
# backlog为int整数，表示允许的连接数量，超出的会等待，可以不填，不填会自动设置一个合理值
```

4. 接收客户端连接，获得连接对象

```
conn, address = socket_server.accept()
print(f"接收到客户端连接，连接来自：{address}")
# accept方法是阻塞方法，如果没有连接，会卡再当前这一行不向下执行代码
# accept返回的是一个二元元组，可以使用上述形式，用两个变量接收二元元组的2个元素
```

Socket服务端编程

5. 客户端连接后，通过recv方法，接收客户端发送的消息

```
while True:
    data = conn.recv(1024).decode("UTF-8")
    # recv方法的返回值是字节数组（Bytes），可以通过decode使用UTF-8解码为字符串
    # recv方法的传参是buffsize，缓冲区大小，一般设置为1024即可
    if data == 'exit':
        break
    print("接收到发送来的数据：", data)
# 可以通过while True无限循环来持续和客户端进行数据交互
# 可以通过判定客户端发来的特殊标记，如exit，来退出无限循环
```

6. 通过conn（客户端当次连接对象），调用send方法可以回复消息

```
while True:
    data = conn.recv(1024).decode("UTF-8")
    if data == 'exit':
        break
    print("接收到发送来的数据：", data)

    conn.send("你好呀哈哈哈".encode("UTF-8"))
```

7. conn（客户端当次连接对象）和socket_server对象调用close方法，关闭连接

实现服务端并结合客户端进行测试

```python
import socket

socket_server = socket.socket()
socket_server.bind(("localhost", 8888))
socket_server.listen(1)
conn, address = socket_server.accept()
print(f"接收到客户端连接，连接来自：{address}")
while True:
    data = conn.recv(1024).decode("UTF-8")
    if data == 'exit':
        break
    print("接收到发送来的数据：", data)
    reply = input("请输入回复消息：").encode("UTF-8")
    conn.send(reply)

conn.close()
socket_server.close()
```

下载网络调试助手作为客户端

https://github.com/nicedayzhu/netAssist/releases

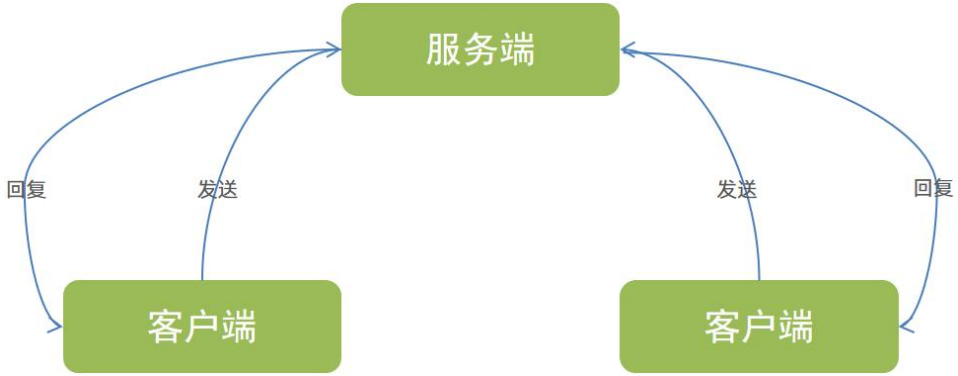
```
接收到客户端连接，连接来自：('127.0.0.1', 2334)
接收到发送来的数据： 你好服务端
请输入回复消息：你好呀客户端
接收到发送来的数据： 今天天气不错呢
请输入回复消息：嗯，明天我们出去玩吧
接收到发送来的数据： 好的，就这么约好啦
请输入回复消息：OK

Process finished with exit code 0
```

总结

1. 什么是Socket

socket（简称 套接字）是进程之间通信一个工具。

2. 什么是服务端、客户端？



目录

Contents

- 闭包
- 装饰器
- 设计模式
- 多线程
- 网络编程
  - 服务端开发
  - 客户端开发
- 正则表达式
- 递归

学习目标

Learning Objectives

1. 基于Socket完成客户端程序开发

Socket客户端编程

主要分为如下几个步骤：

1. 创建socket对象

```
import socket
socket_client = socket.socket()
```

2. 连接到服务端

```
socket_client.connect(("localhost", 8888))
```

3. 发送消息

```
while True:      # 可以通过无限循环来确保持续的发送消息给服务端
    send_msg = input("请输入要发送的消息")
    if send_msg == 'exit':
        # 通过特殊标记来确保可以退出无限循环
        break
    socket_client.send(send_msg.encode("UTF-8"))     # 消息需要编码为字节数组（UTF-8编码）
```

Socket客户端编程

4. 接收返回消息

```
while True:
    send_msg = input("请输入要发送的消息").encode("UTF-8")
    socket_client.send(send_msg)

    recv_data = socket_client.recv(1024)        # 1024是缓冲区大小，一般1024即可
    # recv方法是阻塞式的，即不接收到返回，就卡在这里等待
    print("服务端回复消息为：", recv_data.decode("UTF-8"))   # 接受的消息需要通过UTF-8解码为字符串
```

5. 关闭链接

```
socket_client.close()   # 最后通过close关闭链接
```

服务端客户端相互通讯

结合上一节学习的服务端代码，以及当前学习的客户端代码。

两者均运行起来，进行相互通讯。

```
接收到客户端连接，连接来自：('127.0.0.1', 3358)
接收到发送来的数据：  你好呀
请输入回复消息：你好呀，天气不错
接收到发送来的数据：  是的呢，明天见
请输入回复消息：好的，明天见
```

```
请输入要发送的消息：你好呀
服务端回复消息为：  你好呀，天气不错
请输入要发送的消息：是的呢，明天见
服务端回复消息为：  好的，明天见
请输入要发送的消息：exit
```

目录

Contents

- 闭包
- 装饰器
- 设计模式
- 多线程
- 网络编程
- 正则表达式
  - 基础匹配
  - 元字符匹配
- 递归

学习目标

Learning Objectives

1. 了解什么是正则表达式
2. 掌握re模块的基础使用

正则表达式

正则表达式，又称规则表达式（Regular Expression），是使用单个字符串来描述、匹配某个句法规则的字符串，常被用来检索、替换那些符合某个模式（规则）的文本。

简单来说，正则表达式就是使用：字符串定义规则，并通过规则去验证字符串是否匹配。

比如，验证一个字符串是否是符合条件的电子邮箱地址，只需要配置好正则规则，即可匹配任意邮箱。

比如通过正则规则：`(^[\w-]+(\.[\w-]+)*@[\w-]+(\.[\w-]+)+$)` 即可匹配一个字符串是否是标准邮箱格式

但如果不使用正则，使用if else来对字符串做判断就非常困难了。

正则的三个基础方法

Python正则表达式，使用re模块，并基于re模块中三个基础方法来做正则匹配。

分别是：match、search、findall 三个基础方法

- re.match(匹配规则， 被匹配字符串)

从被匹配字符串开头进行匹配， 匹配成功返回匹配对象（包含匹配的信息），匹配不成功返回空。

```
s = 'python itheima python itheima python itheima'

result = re.match('python', s)
print(result)  # <re.Match object; span=(0, 6), match='python'>
print(result.span())  # (0, 6)
print(result.group())  # python
```

```
s = '1python itheima python itheima python itheima'

result = re.match('python', s)
print(result) # None
```

正则的三个基础方法

- search(匹配规则， 被匹配字符串)

搜索整个字符串，找出匹配的。从前向后，找到第一个后，就停止，不会继续向后

```
s = '1python666itheima666python666'

result = re.search('python', s)
print(result)           # <re.Match object; span=(1, 7), match='python'>
print(result.span())    # (1, 7)
print(result.group())   # python
```

整个字符串都找不到，返回None

```
s = 'itheima666'

result = re.search('python', s)
print(result)           # None
```

正则的三个基础方法

- findall(匹配规则， 被匹配字符串)

匹配整个字符串，找出全部匹配项

```
s = '1python666itheima666python666'

result = re.findall('python', s)
print(result)          # ['python', 'python']
```

找不到返回空list：[]

```
s = '1python666itheima666python666'

result = re.findall('itcast', s)
print(result)          # []
```

总结

1. 什么是正则表达式

是一种字符串验证的规则，通过特殊的字符串组合来确立规则

用规则去匹配字符串是否满足

如(^[\w-]+(\.[\w-]+)*@[\w-]+(\.[\w-]+)+$)可以表示为一个标准邮箱的格式

2. re模块的三个主要方法

- re.match，从头开始匹配，匹配第一个命中项
- re.search，全局匹配，匹配第一个命中项
- re.findall，全局匹配，匹配全部命中项

目录

Contents

- 闭包
- 装饰器
- 设计模式
- 多线程
- 网络编程
- 正则表达式
  - 基础匹配
  - 元字符匹配
- 递归

学习目标

Learning Objectives

1. 掌握正则表达式的各类元字符规则
2. 了解字符串的r标记的作用

元字符匹配

在刚刚我们只是进行了基础的字符串匹配，正则最强大的功能在于元字符匹配规则。

单字符匹配：

字符	功能
.	匹配任意1个字符（除了\n），\. 匹配点本身
[]	匹配[]中列举的字符
\d	匹配数字，即0-9
\D	匹配非数字
\s	匹配空白，即空格、tab键
\S	匹配非空白
\w	匹配单词字符，即a-z、A-Z、0-9、_
\W	匹配非单词字符

示例：

字符串 s = “itheima1 @@python2 !!666 ##itcast3”

- 找出全部数字： re.findall(r‘\d’, s)

字符串的r标记，表示当前字符串是原始字符串，即内部的转义字符无效而是普通字符

- 找出特殊字符：

re.findall(r‘\W’, s)

- 找出全部英文字母：

re.findall(r’[a-zA-Z]’, s)

[]内可以写：[a-zA-Z0-9] 这三种范围组合或指定单个字符如[aceDFG135]

元字符匹配

数量匹配：

字符	功能
*	匹配前一个规则的字符出现0至无数次
+	匹配前一个规则的字符出现1至无数次
?	匹配前一个规则的字符出现0次或1次
{m}	匹配前一个规则的字符出现m次
{m,}	匹配前一个规则的字符出现最少m次
{m,n}	匹配前一个规则的字符出现m到n次

元字符匹配

边界匹配：

字符	功能
^	匹配字符串开头
$	匹配字符串结尾
\b	匹配一个单词的边界
\B	匹配非单词边界

分组匹配：

字符	功能
|	匹配左右任意一个表达式
()	将括号中字符作为一个分组

案例

- 匹配账号，只能由字母和数字组成，长度限制6到10位

规则为：^[0-9a-zA-Z]{6, 10}$

- 匹配QQ号，要求纯数字，长度5-11，第一位不为0

规则为：^[1-9][0-9]{4, 10}&

[1-9]匹配第一位，[0-9]匹配后面4到10位

- 匹配邮箱地址，只允许qq、163、gmail这三种邮箱地址

规则为：^[\w-]+(\.[\w-]+)*@(qq|163|gmail)(\.[\w-]+)+&

- [\w-]+ 表示出现a-z A-Z 0-9 _ 和 - 字符最少一个，最多不限
- (\.[\w-]+)*，表示出现组合 . 和 a-z A-Z 0-9 _ -的组合最少0次，最多不限

用于匹配：abc.ced.efg@123.com中的ced.efg这部分

- @表示匹配@符号
- (qq|163|gmail)表示只匹配这3个邮箱提供商
- (\.[\w-]+)+表示a-z A-Z 0-9 _ -的组合最少1次，最多不限

用于匹配abc.ced.efg@123.com.cn中的.com.cn这种

最后使用+表示最少一次，即比如：.com

多了可以是：.com.cn.eu这样

总结

1. 字符串的r标记表示，字符串内转移字符无效，作为普通字符使用

字符	功能
.	匹配任意1个字符（除了\n），\. 匹配点本身
[]	匹配[]中列举的字符
\d	匹配数字，即0-9
\D	匹配非数字
\s	匹配空白，即空格、tab键
\S	匹配非空白
\w	匹配单词字符，即a-z、A-Z、0-9、_
\W	匹配非单词字符

字符	功能
*	匹配前一个规则的字符出现0至无数次
+	匹配前一个规则的字符出现1至无数次
?	匹配前一个规则的字符出现0次或1次
{m}	匹配前一个规则的字符出现m次
{m,}	匹配前一个规则的字符出现最少m次
{m,n}	匹配前一个规则的字符出现m到n次

字符	功能
^	匹配字符串开头
$	匹配字符串结尾
\b	匹配一个单词的边界
\B	匹配非单词边界

字符	功能
|	匹配左右任意一个表达式
()	将括号中字符作为一个分组

目录

Contents

- 闭包
- 装饰器
- 设计模式
- 多线程
- 网络编程
- 正则表达式
- 递归

学习目标

Learning Objectives

1. 掌握什么是递归
2. 掌握递归案例的开发

递归

递归在编程中是一种非常重要的算法

递归： 即方法（函数）自己调用自己的一种特殊编程写法

如：

```
def func():

    if ...:
        func()

    return ...
```

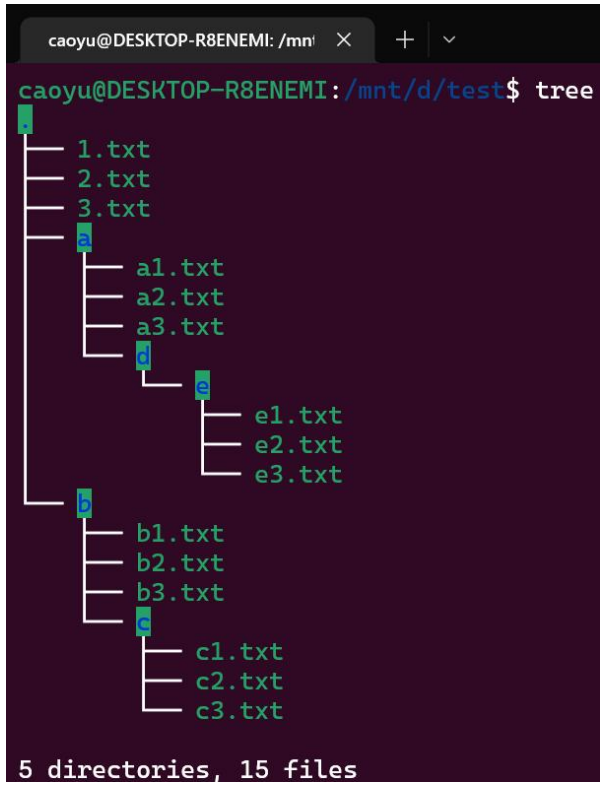
函数调用自己，即称之为递归调用。

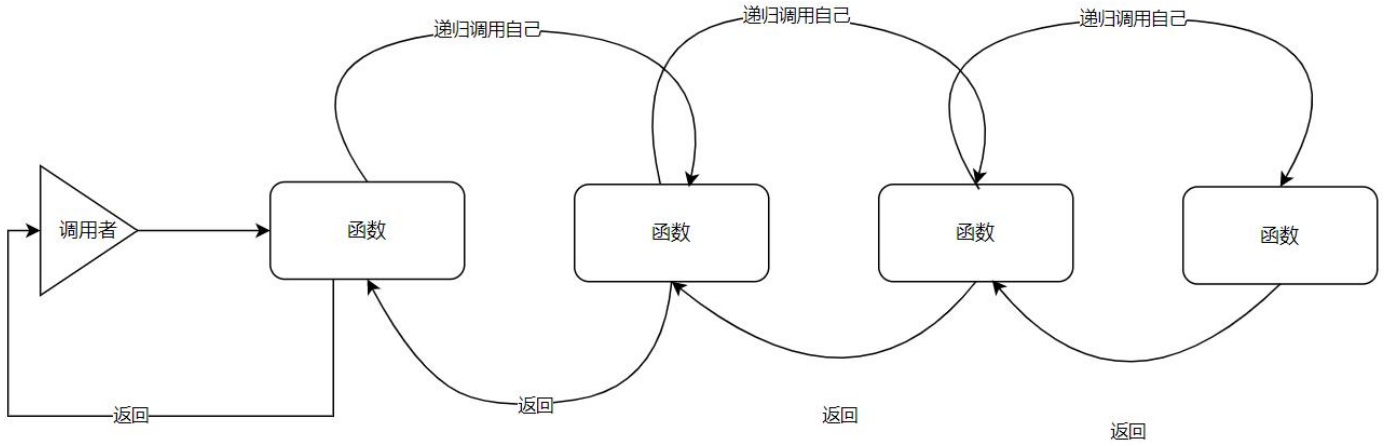
那么，什么场景中会使用到递归呢？

递归找文件

最典型的递归场景为找出一个文件夹中全部的文件。

如图，在D:/test 文件夹内，有如下嵌套结构和所属的文件， 可以通过递归编程的形式完成





让我们进入代码中体会递归的神奇吧

总结

1. 什么是递归

在满足条件的情况下，函数自己调用自己的一种特殊编程技巧

2. 递归需要注意什么？

- 注意退出的条件，否则容易变成无限递归
- 注意返回值的传递，确保从最内层，层层传递到最外层

3. os模块的3个方法

- os.listdir，列出指定目录下的内容
- os.path.isdir，判断给定路径是否是文件夹，是返回True，否返回False
- os.path.exists，判断给定路径是否存在，存在返回True，否则返回False

黑马程序员
www.itheima.com
传智教育旗下高端IT教育品牌